

Plano de Aula — Como desenvolver aplicações web na arquitetura MVC

1. Identificação

- **Tema:** Como desenvolver aplicações web na arquitetura MVC
- **Duração:** 30 minutos
- **Modalidade:** Aula-teste prática (remota, via Teams)
- **Público-alvo (simulado):** Alunos do curso Técnico em Desenvolvimento de Sistemas
- **Pré-requisitos dos alunos:** noções de HTML, JavaScript e o que é um banco de dados
- **Processo:** SENAI-SP — Instrutor de Formação Profissional III - TI (Desenvolvimento de Sistemas) — nº 104-2026
- **Unidade:** Escola SENAI "Nadir Dias de Figueiredo" — Osasco/SP
- **Repositório do projeto usado na aula:** <https://github.com/johnnysoutodev/mvc-app-anotacoes>
- **Data da aula-teste:** 25/09/2026, 16h30

2. Situação de Aprendizagem (contextualização)

Uma pequena empresa usa um app de anotações feito "às pressas": todo o código (tela, regras e dados) está em um único arquivo. Cada alteração quebra outra parte do sistema. A equipe de desenvolvimento (a turma) foi contratada para **organizar o sistema em MVC** e **desenvolver uma nova funcionalidade — excluir nota** — seguindo essa arquitetura.

Essa situação dá sentido prático à aula: o aluno não só aprende o conceito, mas **desenvolve** uma funcionalidade real dentro da arquitetura.

3. Objetivo Geral

Capacitar o aluno a **desenvolver uma funcionalidade de uma aplicação web seguindo a arquitetura MVC**, organizando o código em Model, View e Controller e integrando as camadas do banco de dados até a tela.

4. Capacidades a serem desenvolvidas

Capacidades técnicas

1. Identificar a responsabilidade de cada camada (Model, View, Controller) em uma aplicação web.
2. Organizar a estrutura de pastas de um projeto web no padrão MVC.
3. Implementar uma funcionalidade completa passando pelas camadas: Model → Controller → Rota → View.
4. Reconhecer as vantagens do MVC (manutenção, testes, trabalho em equipe) comparando com um código sem separação.

Capacidades socioemocionais

- Raciocínio lógico e organização.
- Trabalho em equipe (entender que cada camada pode ser desenvolvida por pessoas diferentes).

5. Conhecimentos (conteúdo programático)

- Arquitetura de software: por que organizar o código
- Padrão MVC: Model, View e Controller e o fluxo de uma requisição HTTP
- Estrutura de pastas de um projeto MVC com Node.js + Express + PostgreSQL
- Rotas HTTP (GET, POST, DELETE) como porta de entrada para o Controller
- Passo a passo para desenvolver uma nova funcionalidade em MVC

6. Estratégia de Ensino — distribuição dos 30 minutos

Etapa	Tempo	O que acontece	Recurso
1. Abertura e problema	3 min	Pergunta disparadora e a dor da manutenção + mostrar rapidamente o <code>app-sem-mvc/server.js</code> (monólito sem organização)	Slides 1–3, VS Code
2. Conceito essencial	7 min	A solução MVC, papel de cada camada (com analogia do restaurante), fluxo de uma requisição e vantagens	Slides 4–11
3. Prática: desenvolvendo em MVC	12 min	Tour pela estrutura do <code>app-mvc</code> e desenvolvimento ao vivo da funcionalidade "Excluir nota" , camada por camada	Slides 12–17, VS Code, navegador
4. Quiz de verificação	5 min	5 perguntas no Kahoot (ou similar)	Slide 18 + Kahoot
5. Síntese e fechamento	3 min	Recapitular a "receita" de 5 passos + desafio para casa ("Editar nota") + referências para estudo	Slides 19–20

Detalhamento da prática (etapa 3 — 12 min)

3.1 Tour pela estrutura (2 min) — mostrar no VS Code:

```

app-mvc/
├── db/init.sql           → estrutura da tabela (banco)
├── src/
│   ├── config/db.js     → conexão com o banco
│   ├── models/Nota.js   → MODEL (dados + regras)
│   ├── controllers/notasController.js → CONTROLLER (orquestra)
│   ├── routes/notasRoutes.js → ROTAS (URL → Controller)
│   └── app.js            → inicia o servidor
└── public/              → VIEW (HTML, CSS, JS da tela)

```

Rodar o app e criar uma nota para mostrar funcionando.

3.2 Desenvolvimento ao vivo: "Excluir nota" (9 min) — seguir a receita. Antes de cada passo, lançar a pergunta de forma retórica — *"em qual camada isso entra? No Model!"* — e já responder, para manter a turma engajada sem perder tempo:

Passo	Camada	Arquivo	O que escrever
1	Model	src/models/Nota.js	Método <code>remove(id)</code> : valida o id (regra) e executa o <code>DELETE</code> no banco
2	Controller	src/controllers/notasController.js	Função <code>remove</code> que lê o id da requisição, chama o Model e responde 204 (ou 400 no catch)
3	Rota	src/routes/notasRoutes.js	<code>router.delete("/notas/:id", controller.remove)</code>
4	View	public/script.js	Botão "Excluir" em cada nota, que chama <code>DELETE /api/notas/:id</code> e recarrega a lista
5	Teste	Navegador	Criar uma nota, excluir e ver a lista atualizar

Preparação antes da aula: o repositório já contém a versão final do Model, Controller e Rota de `remove` (código de referência). Para a demonstração ao vivo, esses trechos são removidos em uma branch de ensaio (ex.: `git checkout -b aula-ao-vivo`) e reescritos durante a aula. Código do passo 4 (View) está no slide 16 de `conteudo-slides-mvc.md`.

3.3 Pergunta de verificação (1 min, slide 17) — única pausa para a turma responder: *"A regra 'as favoritas aparecem primeiro' está na View ou no Model?"* (Resposta: Model — o `ORDER BY favorita DESC` em `Nota.js`; abrir o arquivo e mostrar a linha.) A regra já existe no código, então não há nada novo a programar. Se ninguém responder em ~10 segundos, dar a resposta e seguir.

7. Recursos Didáticos

- Slides (PowerPoint) — arquivo `aula-mvc.pptx` (PDF: `aula-mvc.pdf`); textos e falas em `conteudo-slides-mvc.md`
- VS Code com o projeto `app-mvc` aberto e o banco rodando (`docker compose up -d`)
- Navegador em `http://localhost:3000`
- Projeto `app-sem-mvc` para contraste
- Quiz online gratuito (Kahoot, Wayground/Quizizz ou Google Forms)
- Compartilhamento de tela via Teams

8. Avaliação (verificação de aprendizagem)

Quiz (5 min) — instrumento principal — 5 perguntas objetivas, 4 alternativas cada, no Kahoot. Meta: pelo menos 80% de acertos na turma. O ranking do Kahoot mostra na hora quem acompanhou.

Pergunta de verificação (1 min) — uma única pergunta oral durante a prática (item 3.3), para checar se o aluno sabe em qual camada colocar uma nova regra.

Critérios de avaliação

Critério	Tipo	Evidência
Identifica a responsabilidade de Model, View e Controller	Crítico	Quiz (perguntas 1–4)
Identifica em qual camada fica uma regra de negócio	Crítico	Pergunta de verificação (item 3.3)
Reconhece as vantagens do MVC em relação ao código sem separação	Desejável	Quiz (pergunta 5)
Participa e interage durante a construção ao vivo	Desejável	Observação do instrutor

9. Quiz — 5 perguntas para o site de quiz

Sugestão de site gratuito: [Kahoot](#) (plano gratuito, alunos entram pelo celular com PIN). Alternativas: [Wayground](#) (antigo Quizizz), [Mentimeter](#) ou Google Forms em modo teste. **Configuração sugerida:** 20 segundos por pergunta, tipo "Quiz" com resposta única. A alternativa correta está marcada com  (mudei a posição dela em cada pergunta).

Pergunta 1 — O que significa a sigla MVC?

- A) Model, View, Controller 
- B) Muito Vídeo Colorido
- C) Mouse, Volume e Carregador
- D) Máquina Voadora de Café

Pergunta 2 — No MVC, qual camada guarda os dados e as regras de negócio (ex.: "o título da nota é obrigatório")?

- A) O papel de parede do computador
- B) A caixa de som
- C) Model 
- D) O teclado sem fio

Pergunta 3 — Qual camada é responsável por mostrar as informações na tela para o usuário?

- A) O cabo de energia
- B) View 
- C) O ventilador do processador
- D) A garantia do monitor

Pergunta 4 — Qual é o papel do Controller?

- A) Controlar o volume da televisão
- B) Ligar e desligar o ar-condicionado
- C) Trocar o canal do videogame
- D) Receber a requisição do usuário, acionar o Model e devolver a resposta 

Pergunta 5 — Qual é a principal vantagem de desenvolver uma aplicação web em MVC?

- A) Separar responsabilidades, facilitando manutenção, testes e trabalho em equipe 
- B) Fazer o computador funcionar sem energia elétrica

- C) Deixar a internet mais rápida em todo o bairro
- D) Não precisar mais escrever código nenhum

10. Referências

Fontes originais do padrão

- REENSKAUG, Trygve. *MVC — Xerox PARC 1978-79* (notas originais do criador do MVC). Disponível em: <https://folk.universitetetioslo.no/trygver/themes/mvc/mvc-index.html>
- KRASNER, Glenn E.; POPE, Stephen T. *A Cookbook for Using the Model-View-Controller User Interface Paradigm in Smalltalk-80*. Journal of Object-Oriented Programming, v. 1, n. 3, p. 26-49, 1988.

Livros

- GAMMA, Erich; HELM, Richard; JOHNSON, Ralph; VLISSIDES, John. *Padrões de Projeto: soluções reutilizáveis de software orientado a objetos*. Porto Alegre: Bookman, 2000. (Capítulo 1 apresenta o MVC.)
- FOWLER, Martin. *Padrões de Arquitetura de Aplicações Corporativas*. Porto Alegre: Bookman, 2006. (Padrões Model View Controller, Page Controller e Front Controller.)
- SOMMERVILLE, Ian. *Engenharia de Software*. 10. ed. São Paulo: Pearson, 2018. (Capítulo 6 — Projeto de arquitetura: padrão MVC.)
- PRESSMAN, Roger S.; MAXIM, Bruce R. *Engenharia de Software: uma abordagem profissional*. 8. ed. Porto Alegre: AMGH, 2016. (Projeto de arquitetura de aplicações web.)
- BUSCHMANN, Frank et al. *Pattern-Oriented Software Architecture, Volume 1: A System of Patterns*. Chichester: Wiley, 1996. (Padrão Model-View-Controller.)

Documentação oficial (on-line)

- MDN Web Docs — MVC: <https://developer.mozilla.org/pt-BR/docs/Glossary/MVC>
- FOWLER, Martin. *GUI Architectures*: <https://martinfowler.com/eaDev/uiArchs.html>
- Express.js — Roteamento: <https://expressjs.com/pt-br/guide/routing.html>
- Microsoft Learn — Visão geral do ASP.NET Core MVC: <https://learn.microsoft.com/pt-br/aspnet/core/mvc/overview>
- Spring Framework — Web MVC: <https://docs.spring.io/spring-framework/reference/web/webmvc.html>